

Laboratorio: Buscaminas Distribuido TCP

Computación en Internet I (09810 - TIC) · Período 2026-2 · NRC 12378 / Grupo 001 Semana 08 /
Sesión 16 — Sockets TCP, Serialización JSON y ThreadPools

Estudiante 1	Bryan Díaz
Código	_____
Estudiante 2	_____
Código	_____
Fecha	24 de septiembre de 2026
Política IAG	Nivel 3 — Colaboración Asistida

1. Delimitación de Mensajes en TCP

1.1 Qué es el *TCP framing*

TCP entrega un **flujo continuo de octetos** fiable y ordenado, pero **no preserva los límites de escritura de la aplicación**. El estándar (RFC 793) define TCP como un servicio *byte-stream*: la capa de transporte garantiza que los bytes lleguen completos y en orden, pero no que lleguen agrupados igual que como se escribieron. Tres mecanismos rompen esa correspondencia:

- **Segmentación por MSS:** un `write()` de 4 KB se parte en varios segmentos si supera el *Maximum Segment Size* negociado.
- **Coalescencia / algoritmo de Nagle:** varios `write()` pequeños y consecutivos se acumulan en el búfer de envío del kernel y viajan en un único segmento.
- **Buffering en recepción:** el receptor entrega al `read()` lo que haya disponible en ese instante, que puede ser medio mensaje o dos mensajes y medio.

En consecuencia, la relación entre escrituras y lecturas es **N:M, no 1:1**. Es la capa de aplicación la que debe definir un esquema de *framing* (delimitación de tramas). Las tres estrategias clásicas son:

1. **Longitud fija** — todos los mensajes ocupan el mismo número de bytes.
2. **Prefijo de longitud** — se envía primero un entero con el tamaño del payload (lo que hacen gRPC o el protocolo de MySQL).
3. **Delimitador** — un octeto centinela marca el fin de la trama, que es la estrategia de este laboratorio (y la de HTTP/1.1 con `\r\n`, SMTP o Redis RESP).

Este protocolo usa **delimitación por salto de línea**: cada mensaje es un JSON de una sola línea terminado en `\n`. El receptor lo recompone con `BufferedReader.readLine()`, que acumula octetos hasta encontrar `\n`, `\r` o el fin de flujo (EOF).

1.2 Qué falla si se omite el `\n`

`readLine()` es una operación **bloqueante que no retorna hasta encontrar un terminador**. Si el cliente envía `{"action":"GET_BOARD","data":{}}` sin el salto de línea:

1. Los bytes llegan al búfer de recepción del servidor.
2. `readLine()` los consume, no encuentra `\n` y **se vuelve a bloquear** esperando más datos.
3. El cliente, que ya escribió su petición, entra en su propio `readLine()` esperando la respuesta.

El resultado es un **interbloqueo distribuido**: ninguna de las dos partes escribirá nada más y ninguna cerrará el socket. La conexión queda colgada indefinidamente porque, al no haber `SO_TIMEOUT` configurado, no existe ningún temporizador que la rompa. Solo se desbloquearía si el cliente cerrara el socket: entonces el servidor recibiría un FIN, `readLine()` devolvería el texto acumulado como si fuera una línea y el flujo continuaría — pero para entonces ya no hay a quién responderle.

Un segundo efecto aparece si se envían **dos peticiones seguidas sin delimitador**: llegan concatenadas como `{...}{...}` y `readLine()` las entrega como una sola cadena. `Gson.fromJson` en modo estricto lanzaría `JsonSyntaxException`; en modo laxo deserializaría solo el primer objeto y descartaría el segundo silenciosamente, que es el fallo más difícil de diagnosticar.

1.3 Qué falla si se omite el `flush()`

`BufferedWriter` mantiene un búfer intermedio en memoria de la JVM (8192 caracteres por defecto) cuyo propósito es **evitar una llamada al sistema por cada carácter escrito**. Los métodos `write()` y `newLine()` **solo escriben en ese búfer de usuario**; no tocan el socket.

Una petición típica de este protocolo ocupa unos 50–60 bytes, muy por debajo del umbral de 8192. Sin `flush()` el JSON **nunca abandona el espacio de memoria del proceso cliente**: no se invoca `write(2)` sobre el descriptor, no se copia nada al búfer de envío del kernel y no se emite ningún segmento TCP. Desde el punto de vista del servidor no ha llegado absolutamente nada, así que `readLine()` se bloquea sobre un flujo vacío y se reproduce exactamente el mismo interbloqueo de la sección anterior.

Es importante distinguir las tres capas de *buffering* que atraviesa el mensaje, porque cada una necesita su propio disparador:

Capa	Contenido	Cómo se vacía
<code>BufferedWriter</code> (JVM)	caracteres	<code>flush()</code>
Búfer de envío del socket (kernel)	octetos	el kernel, según ventana y Nagle
Red	segmentos TCP	la NIC

`newLine()` únicamente inserta el delimitador **dentro** del búfer de la JVM. Por eso las tres llamadas del cliente son necesarias y en ese orden: `write(json)` deposita el payload, `newLine()` añade el centinela de trama y `flush()` empuja ambos hacia el kernel.

Como mitigación defensiva, en un sistema de producción se añadiría `socket.setSoTimeout(milisegundos)` para que `readLine()` lance `SocketTimeoutException` en lugar de bloquearse para siempre, y un límite máximo de longitud de línea para evitar que un cliente malicioso agote la memoria del servidor enviando una línea infinita.

2. Ventajas y Sobrecargas de los ThreadPools

2.1 Ventajas frente a `new Thread()` por cliente

a) Acotación del consumo de recursos. Cada hilo de plataforma en Java es un hilo del sistema operativo y reserva una pila propia (`-Xss` , 512 KB–1 MB por defecto según plataforma) más las estructuras del kernel. El esquema *thread-per-client* es **no acotado**: el número de hilos lo decide el cliente, no el servidor. Mil conexiones simultáneas significan aproximadamente un gigabyte solo en pilas y, con frecuencia, un `OutOfMemoryError: unable to create native thread`. Un pool fijo convierte ese consumo en una constante conocida en tiempo de diseño: 5 hilos.

b) Amortización del costo de creación. Crear un hilo implica una llamada al sistema (`clone` / `pthread_create`), reservar y mapear la pila y registrarlo en el planificador; cuesta del orden de decenas de microsegundos. En este protocolo cada conexión atiende **un solo comando** que se resuelve en microsegundos, de modo que el costo de crear y destruir el hilo dominaría el tiempo total de servicio. El pool paga ese costo cinco veces durante toda la vida del proceso.

c) Menor conmutación de contexto y mejor localidad de caché. Con más hilos ejecutables que núcleos, el planificador reparte el CPU en rodajas y cada cambio invalida caché L1/L2 y provoca fallos de TLB. Es el fenómeno de *thrashing*: el sistema dedica una fracción creciente del CPU a administrarse a sí mismo. Mantener el número de trabajadores cercano al número de núcleos maximiza el trabajo útil.

d) Control de admisión implícito. El pool actúa como limitador natural de la carga concurrente: por muchas conexiones que lleguen, nunca habrá más de cinco ejecutándose a la vez. Esto acota también la contención sobre el `BoardGame` compartido.

e) **Separación entre política y mecanismo.** `ExecutorService` desacopla *qué* se ejecuta (`Runnable`) de *cómo* se ejecuta, y permite apagado ordenado (`shutdown()` / `awaitTermination()`), que el esquema de hilos sueltos no ofrece.

2.2 Qué ocurre con 6 o más clientes simultáneos

No se rechaza ninguna conexión ni se lanza excepción alguna. La secuencia es la siguiente:

1. El *handshake* de tres vías lo completa **el kernel**, no la aplicación. Las conexiones establecidas se depositan en la *accept queue* del `ServerSocket`, cuyo tamaño es el `backlog` (aquí 50). Un cliente puede conectarse y enviar su petición aunque la aplicación todavía no lo haya aceptado.
2. El hilo principal ejecuta `serverSocket.accept()` en bucle y envuelve cada socket en un `TCPClientHandler`, que entrega al `executor`.
3. `Executors.newFixedThreadPool(5)` se construye sobre un `ThreadPoolExecutor` con `corePoolSize = maximumPoolSize = 5` y una **`LinkedBlockingQueue` no acotada**. Cuando los cinco hilos están ocupados, las tareas 6, 7, 8... **se encolan** y esperan.
4. En cuanto un trabajador termina su tarea, toma la siguiente de la cola.

Por tanto, el efecto sobre el sexto cliente es **latencia adicional, no error**: espera el tiempo que tarde en liberarse un trabajador. Como cada comando es corto (parsear, mutar el tablero, serializar), en la práctica la cola se drena en microsegundos y **cinco hilos sirven a decenas de jugadores sin degradación perceptible**. Esto es consecuencia directa del diseño de conexión corta: ningún hilo se queda esperando a que un humano piense.

Hay dos sobrecargas que conviene declarar explícitamente:

- **Cola no acotada = fuga de memoria bajo inundación.** Si la tasa de llegada supera sostenidamente la de servicio, la `LinkedBlockingQueue` crece sin límite hasta agotar el *heap*. Un diseño de producción usaría `new ThreadPoolExecutor(5, 5, 0L, MILLISECONDS, new ArrayBlockingQueue<>(100), new ThreadPoolExecutor.CallerRunsPolicy())` para ejercer contrapresión.
- **Riesgo de inanición si las tareas bloquean.** El pool fijo asume tareas de corta duración. Si una tarea bloqueara durante toda la partida (ver pregunta 4), cinco jugadores agotarían el pool y el sexto quedaría en cola indefinidamente. Es el escenario clásico de *thread starvation deadlock*.

3. Condiciones de Carrera y Exclusión Mutua

El servidor mantiene **una única instancia compartida de `BoardGame`** (creada en el constructor de `ServicesImpl` y pasada a todos los `TCPClientHandler`). Sin `synchronized`, dos clientes que ejecutan `selectCell` sobre celdas contiguas en el mismo milisegundo son atendidos por dos hilos distintos del pool que escriben concurrentemente sobre la misma matriz `Cell[][]`. Los problemas concretos son los siguientes.

3.1 Violación de la invariante de terminación del *flood fill*

`selectCell` delega en `showCells(i, j, true)`, un recorrido recursivo en profundidad sobre las 8 celdas vecinas. Su condición de parada es:

```
if (i < 0 || i >= board.length || j < 0 || j >= board[0].length || !board[i][j].isHide())
{
    return;
}
if (deep && board[i][j].isHide()) {
    board[i][j].setHide(false);
    deep = board[i][j].getValue() == 0;
}
```

El algoritmo usa el campo `hide` como **marca de visitado**: "si ya está destapada, no la vuelvas a recorrer". Esa marca es la única garantía de que la recursión termina en un grafo con ciclos (cada celda es vecina de sus vecinas). El problema es que **comprobar `isHide()` y ejecutar `setHide(false)` no es una operación atómica**: entre ambas hay una ventana en la que otro hilo puede colarse.

En celdas contiguas el solapamiento es máximo, porque las dos regiones de expansión se tocan:

- Los hilos A y B llegan a la misma celda X con `hide == true`.
- Ambos superan la guarda.
- Ambos ejecutan `setHide(false)` y ambos descienden sobre los 8 vecinos de X.

El recorrido se **duplica exponencialmente** en la zona de solape, multiplicando el trabajo y la profundidad de pila. La versión benigna del fallo es una respuesta lenta; la grave es `StackOverflowError`, que mata al hilo del pool y deja al cliente sin respuesta (su `readLine()` recibe EOF).

3.2 Ausencia de barrera de memoria (visibilidad)

Este es el problema más sutil y el que hace que el anterior no sea meramente probabilístico. El *Java Memory Model* solo garantiza que un hilo observe las escrituras de otro si existe una relación **happens-before** entre ellas. Sin `synchronized`, `volatile` ni ninguna otra barrera, no hay ninguna: `hide` es un `boolean` común.

Las consecuencias son concretas:

- El hilo B puede leer indefinidamente el valor **obsoleto** `hide == true` para una celda que A ya destapó, porque el valor vive en la caché del núcleo de A o en un registro.
- El compilador JIT está **autorizado** a izar (`hoist`) la lectura `board[i][j].isHide()` fuera del recorrido y reutilizar el valor cacheado, precisamente porque puede asumir ausencia de concurrencia sobre un campo no sincronizado.

Si la marca de visitado deja de ser observable, **la recursión pierde su condición de parada** y el `StackOverflowError` pasa de improbable a esperable. Dicho de otro modo: la corrección de un algoritmo cuya terminación depende de observar la escritura de otro hilo es imposible de garantizar sin una barrera de memoria.

3.3 Lectura inconsistente en `validWin()`

`selectCell` termina llamando a `validWin()`, que recorre el tablero completo comprobando `!hide || isLandMine` para cada celda. Mientras ese bucle avanza, el otro hilo sigue destapando celdas. El resultado es una **fotografía desgarrada** (*torn read*): la primera mitad del tablero se lee en un estado y la segunda en otro. Se puede reportar `win = true` cuando aún quedaban celdas ocultas, o `win = false` justo después de que el otro jugador completara la partida.

3.4 Operación compuesta no atómica en `markCell`

```
cell.setMarked(!cell.isMarked());
```

Es un **read-modify-write** clásico. Dos clientes que alternan la bandera de la misma celda pueden leer ambos `false` y escribir ambos `true`: se pierde una de las dos operaciones y el resultado observable es que un clic "no hizo nada". Es el mismo patrón que hace incorrecto `i++` en concurrencia.

3.5 Publicación insegura en `initGame`

`initGame` ejecuta `board = new Cell[n][m]` y **después** llena la matriz en un bucle. Un hilo que esté ejecutando `selectCell` durante esa ventana puede:

- leer la **referencia nueva** con celdas todavía en `null` → `NullPointerException`;
- conservar índices válidos para el tablero anterior que quedan fuera de rango en el nuevo → `ArrayIndexOutOfBoundsException`.

Ambas excepciones escapan del `switch` y el cliente recibe una respuesta vacía o ninguna.

3.6 Solución aplicada

Se marcaron como `synchronized` los métodos de instancia que mutan o leen el estado compartido: `initGame`, `selectCell`, `markCell`, `getBoard` y `getMines`.

Al tratarse de métodos de instancia, todos adquieren **el mismo monitor (this)**, de modo que sobre una misma instancia de `BoardGame` solo puede ejecutarse uno a la vez. Esto resuelve las dos dimensiones del problema simultáneamente:

- **Atomicidad:** la recursión completa de `showCells` y el recorrido de `validWin` se vuelven indivisibles frente a otros hilos.

- **Visibilidad:** la liberación del monitor establece un *happens-before* con su siguiente adquisición, por lo que todo hilo que entre ve las escrituras del anterior. Desaparecen tanto las lecturas obsoletas como las reordenaciones del JIT.

Los métodos privados `showCells`, `getMinesAround` y `validWin` **no requieren sincronización propia**: solo se alcanzan desde métodos públicos ya sincronizados y los monitores de Java son **reentrant**, así que la llamada anidada conserva el candado ya adquirido.

Se eligió deliberadamente un **candado de grano grueso** (toda la instancia) en lugar de uno por celda. Un candado por celda obligaría a adquirir hasta nueve monitores en cada paso de la recursión y, sin un orden global de adquisición, introduciría riesgo de interbloqueo. Dado que cada operación dura microsegundos y el pool solo tiene cinco hilos, la contención es despreciable frente a la complejidad que se evita.

4. Conexiones Cortas vs. Conexiones Persistentes

En el diseño actual el servidor cierra el socket después de responder cada comando, de forma que una partida de 30 jugadas abre y cierra 30 conexiones TCP.

4.1 Consumo de descriptores de archivo

Conexión corta. El descriptor existe únicamente durante la ejecución de un comando (del orden de microsegundos). El pico de descriptores en el servidor es proporcional al número de **comandos en vuelo**, no al número de jugadores; con cinco hilos el pico efectivo es de unos pocos. Hay sin embargo un costo oculto: **el extremo que cierra primero entra en estado `TIME_WAIT` durante $2 \times \text{MSL}$** (unos 60 segundos). Bajo una tasa alta de comandos se acumulan miles de sockets en `TIME_WAIT`, y como el que cierra activamente puede ser el cliente, **es el cliente quien arriesga agotar su rango de puertos efímeros** (~28 000 en Linux). En el servidor, además, se consume una entrada de la tabla de conexiones por cada socket en `TIME_WAIT` aunque ya no haya descriptor asociado.

Conexión persistente. Un descriptor por jugador durante toda la partida, más los búferes de envío y recepción del kernel asociados (decenas de kilobytes cada uno). Doscientos jugadores son doscientos descriptores permanentes, lo que obliga a elevar `ulimit -n` y a dimensionar la memoria del kernel. A cambio desaparece por completo la acumulación de `TIME_WAIT`.

4.2 Latencia

Conexión corta. Cada comando paga el **handshake de tres vías completo, es decir 1 RTT extra**, más las llamadas al sistema `socket()`, `connect()`, `close()` y el cierre de cuatro vías. En una LAN el sobre costo es de unos 0,2 ms y resulta imperceptible, pero sobre una WAN con 60 ms de RTT **la latencia por jugada se duplica**: 60 ms de handshake más 60 ms de petición/respuesta. Se pierde además el estado del control de congestión: cada conexión reinicia en *slow start* con una ventana mínima, y si hubiera TLS habría que renegociar la sesión en cada comando.

Conexión persistente. El handshake se paga una sola vez al iniciar la partida; a partir de ahí cada jugada cuesta **1 RTT**. La conexión conserva su ventana de congestión ya ajustada. Habilita además algo que la conexión corta no puede ofrecer: **notificaciones iniciadas por el servidor** (*server push*), necesarias si se quisiera avisar a un jugador de la jugada de otro. Con conexión corta la única alternativa sería *polling*, que introduce latencia y tráfico inútil.

4.3 Escalabilidad

Aquí la comparación se invierte y es el punto decisivo.

Conexión corta. Encaja de forma natural con un pool fijo pequeño: como ningún hilo permanece bloqueado esperando la decisión de un humano, **los cinco trabajadores pueden servir a cientos de jugadores**. El servidor es efectivamente sin estado por conexión, lo que facilita además colocar un balanceador de carga delante y escalar horizontalmente. Es el modelo de HTTP/1.0.

Conexión persistente sobre esta misma arquitectura. Sería **catastrófica**. Un hilo bloqueado en `readLine()` durante toda la partida consume un trabajador del pool mientras el jugador piensa, de modo que el sistema soportaría exactamente **cinco jugadores concurrentes** y el sexto quedaría encolado indefinidamente. Es el escenario de *thread starvation deadlock* anticipado en la pregunta 2. Migrar a conexión persistente exige cambiar también el modelo de ejecución:

- **Hilos virtuales** (`Executors.newVirtualThreadPerTaskExecutor()`, Java 21+), donde un hilo bloqueado no retiene un hilo de plataforma y el costo por conexión baja a unos cientos de bytes; o
- **E/S no bloqueante** con `java.nio.channels.Selector`, donde un puñado de hilos multiplexa miles de conexiones (modelo Netty).

4.4 Conclusión

Criterio	Conexión corta	Conexión persistente
Descriptores en el servidor	mínimos; riesgo de <code>TIME_WAIT</code>	1 por jugador, permanente
Latencia por comando	2 RTT (handshake + intercambio)	1 RTT
Jugadores concurrentes con pool de 5	cientos	cinco
<i>Server push</i>	imposible	posible
Complejidad	baja	alta (requiere NIO o hilos virtuales)

Para el alcance de este laboratorio la conexión corta es la decisión correcta: maximiza la utilización del pool y mantiene el código simple. La conexión persistente solo se justifica si se añaden notificaciones en tiempo real o si el RTT hacia el cliente es alto, y en ese caso debe ir acompañada de hilos virtuales o

NIO y de estado de sesión por conexión — lo que a su vez resolvería la limitación actual de que todos los clientes comparten un único `BoardGame`.

5. Declaración y Reflexión sobre IAG

5.1 Herramientas utilizadas

Herramienta	Modelo	Uso
Claude Code (CLI)	Claude Opus 5 / Sonnet 5	Explicación conceptual del requisito de sincronización, construcción del submódulo cliente, redacción de este informe

Nivel de política aplicado: **3 — Colaboración Asistida**. La IA generó código y texto que fueron revisados, ejecutados y verificados antes de incorporarse al repositorio.

5.2 Prompts empleados

- "*Explícame este requerimiento y una forma sencilla de implementarlo*", acompañado del texto literal del punto 3.2.4 de la guía sobre tratamiento de concurrencia. Sirvió para identificar las secciones críticas de `BoardGame.java` antes de escribir código.
- "*Con el siguiente documento realizar lo que se pide desde el punto de la fase 3, y génrame el documento donde piden respuesta a las preguntas. Aparte por cada fase realiza 1 o más commits, todo listo para pushearlo yo manualmente. No hagas cambios drásticos.*", adjuntando el PDF de la guía. Produjo el submódulo `client`, los DTO, `BuscaminasTCPClient`, `BoardRenderer`, `MainClient` y este informe.

La restricción explícita de "*no hacer cambios drásticos*" se usó deliberadamente para acotar el alcance de la IA al código base existente y evitar refactorizaciones no solicitadas.

5.3 Validación del código propuesto

No se aceptó ninguna sugerencia sin comprobación. El procedimiento seguido fue:

a) Compilación. `./gradlew build` sobre ambos módulos. Este paso reveló que el código base ya contenía dos errores de compilación propios en el manejador `MARK_CELL` (`case "MARK_CELL";` con punto y coma en vez de dos puntos, y `Integer.parseInt` mal escrito), que la IA no había detectado por inspección y que solo salieron a la luz al compilar. También se detectó una incompatibilidad de entorno ajena al código: Gradle 8.6 no soporta JDK 26 (`Unsupported class file major version 70`), por lo que se fijó JDK 17.

b) Contraste contra la tabla del protocolo. Se verificó una por una que las cinco acciones enviadas por el cliente coincidieran literalmente con el catálogo de la guía — incluido el nombre `SOW_ALL`, que

la IA tendía a "corregir" a `SHOW_ALL` y que habría caído en la rama `default` del `switch` devolviendo una respuesta vacía.

c) Pruebas funcionales deterministas. Se ejercitaron los caminos de fin de partida con tableros contruidos para forzar cada resultado: `1x1` con una mina para la derrota (la única celda es necesariamente la mina) y `2x2` con cero minas para la victoria. Se comprobó también el rechazo de coordenadas fuera de rango y de parámetros de tablero inválidos.

d) Prueba de concurrencia. Tres clientes simultáneos contra el mismo servidor, verificando en la bitácora que los cinco hilos del pool atendieran peticiones intercaladas (anexo A).

e) Revisión de seguridad. Se examinaron explícitamente los siguientes puntos:

- **Deserialización.** `Gson.fromJson` vincula contra clases DTO fijas (`Request`, `Response`, `Cell`) sin polimorfismo ni `TypeAdapter` dinámicos, lo que descarta los ataques de *gadget chain* típicos de la deserialización nativa de Java.
- **Validación de entrada.** Todo dato del protocolo se convierte con `Integer.parseInt` dentro de bloques `try / catch`; los índices los valida el modelo antes de indexar la matriz. No hay concatenación hacia shell, SQL ni rutas de archivo, de modo que no aplican inyección de comandos ni *path traversal*.
- **Superficie de red.** El cambio a `0.0.0.0` es un requisito explícito de la guía, pero se deja constancia de que expone el servicio en todas las interfaces; en una red no confiable debería enlazarse a `127.0.0.1` o protegerse con firewall.
- **Limitaciones conocidas y no corregidas.** `readLine()` sobre un flujo sin límite de longitud permitiría a un cliente malicioso agotar la memoria enviando una línea infinita, y la `LinkedBlockingQueue` no acotada del pool permite crecimiento ilimitado de la cola bajo inundación. Ambas son vulnerabilidades de denegación de servicio reales; se documentan aquí en lugar de corregirse porque exceden el alcance de la guía.

5.4 Reflexión

El aporte más valioso de la IA fue acelerar la escritura del código repetitivo del cliente (DTO, apertura de sockets, renderizado) y articular el análisis conceptual. Su limitación más clara quedó demostrada en el punto (a): **no detecta por lectura errores que solo aparecen al ejecutar**. La compilación y las pruebas siguen siendo la única fuente de verdad, y el criterio para decidir qué tocar y qué no del código base fue una decisión humana que la IA respetó pero no habría tomado por sí sola.

Anexo A — Evidencia de ejecución concurrente

Bitácora del servidor con tres clientes simultáneos ejecutando jugadas intercaladas. Cada línea identifica el hilo del `ThreadPool` que atendió la petición:

```
TCP Service started on port 12345
[pool-1-thread-1] Client connected: /127.0.0.1
[pool-1-thread-1] action=INIT_GAME data={n=9, m=9, minas=10}
[pool-1-thread-1] Client disconnected: /127.0.0.1
[pool-1-thread-2] Client connected: /127.0.0.1
[pool-1-thread-4] Client connected: /127.0.0.1
[pool-1-thread-3] Client connected: /127.0.0.1
[pool-1-thread-2] action=MARK_CELL data={i=0, j=2}
[pool-1-thread-3] action=MARK_CELL data={i=0, j=1}
[pool-1-thread-2] Client disconnected: /127.0.0.1
[pool-1-thread-3] Client disconnected: /127.0.0.1
[pool-1-thread-4] Client disconnected: /127.0.0.1
[pool-1-thread-5] Client connected: /127.0.0.1
[pool-1-thread-5] action=GET_BOARD data={}
[pool-1-thread-5] Client disconnected: /127.0.0.1
[pool-1-thread-3] action=SELECT_CELL data={i=8, j=2}
[pool-1-thread-4] action=SELECT_CELL data={i=8, j=3}
[pool-1-thread-5] action=SELECT_CELL data={i=8, j=1}
[pool-1-thread-2] action=MARK_CELL data={i=1, j=2}
[pool-1-thread-1] action=MARK_CELL data={i=1, j=3}
```

Se observa que los **cinco** hilos del pool (`pool-1-thread-1` ... `pool-1-thread-5`) participan, que las peticiones de distintos clientes se intercalan y que ningún cliente bloquea a los demás.

Nota: el servidor vuelca además el tablero completo a su salida estándar en cada petición (`ServicesImpl.printBoard`). Como `BoardGame.printBoard` usa `System.out.print` sin salto de línea final, ese volcado aparece mezclado con las líneas de bitácora cuando varios hilos escriben a la vez. Las líneas anteriores se muestran filtradas para facilitar la lectura.

Anexo B — Capturas de pantalla

(Insertar aquí las capturas solicitadas por la guía: servidor en ejecución, cliente jugando una partida completa y dos terminales de cliente concurrentes contra el mismo servidor.)